

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia (TUPaD) Cátedra:
Organización Empresarial

TRABAJO PRÁCTICO INTEGRADOR

Automatización del proceso de Alta de Proveedores mediante un chatbot

Integrantes: Escobar Gonzalez Tomás, Fernandez Martina

Docentes: Prof. Gabriela Martínez (titular).

Prof. Carolina Bruno, Prof. Mario Raúl López, Prof. Andrea Ramos, Prof. Alejandro Lencinas (adjuntos).

Fecha de entrega: junio de 2026

1. INTRODUCCIÓN

El presente trabajo integrador tiene como objetivo actuar como consultores tecnológicos de una empresa y resolver un problema concreto: automatizar un proceso administrativo que hoy se realiza de forma manual e ineficiente. Para eso elegimos el proceso de alta de proveedores, modelamos su flujo con la notación estándar BPMN 2.0 y desarrollamos un chatbot (en forma de simulador) que ejecuta ese flujo de punta a punta.

La idea central del trabajo es mostrar la coherencia entre el modelo de negocio y el código: primero entendemos cómo funciona el proceso dentro de la organización (el “qué”) y recién después lo traducimos a una lógica de programación (el “cómo”). De esta manera, cada decisión que aparece en el diagrama tiene su correlato en el programa mediante estructuras condicionales.

2. ANÁLISIS DE LA ORGANIZACIÓN

La empresa elegida es DevSolutions S.R.L., una organización ficticia dedicada al desarrollo de software. Por su actividad, pertenece al sector cuaternario de la economía (generación y gestión de conocimiento y servicios tecnológicos) y es una empresa privada de tamaño mediano constituida como Sociedad de Responsabilidad Limitada.

Siguiendo la Teoría General de Sistemas, podemos ver a la empresa como un sistema abierto: recibe entradas del entorno (pedidos de clientes, servicios de proveedores), las transforma mediante sus procesos internos y entrega salidas (productos de software y servicios). Dentro de su estructura, el proceso que analizamos pertenece al área de Compras, que se encarga de la identificación y selección de proveedores. Aunque no es un proceso central del negocio (no programa software), es un proceso de apoyo imprescindible: sin proveedores bien registrados, las áreas clave no pueden trabajar.

3. SITUACIÓN ACTUAL DEL PROCESO (AS-IS)

Actualmente, cuando un proveedor nuevo quiere trabajar con DevSolutions, el alta se hace de forma manual. El proveedor envía su documentación por correo electrónico, un empleado del área de Compras carga los datos a mano en una planilla, verifica “a ojo” que el CUIT sea correcto y reenvía la información por mail al responsable para que la apruebe.

Este proceso manual genera varios problemas:

- Es lento: depende de que las personas revisen y reenvíen correos.
- Se cargan datos con errores (CUIT mal tipeados, campos incompletos).
- Se generan proveedores duplicados, porque nadie controla si el CUIT ya existe.
- Falta trazabilidad: no queda registro claro de quién aprobó qué y cuándo.

4. PROPUESTA DE SOLUCIÓN (TO-BE)

Proponemos reemplazar el circuito manual por un chatbot que guíe el alta paso a paso. El bot le pide los datos al usuario uno por uno, valida cada dato en el momento (formato y duplicados), consulta la base de datos y, según las reglas de negocio, decide automáticamente el resultado o deriva la solicitud al Responsable de Compras.

Definimos tres reglas de negocio que se traducen en tres puntos de decisión (compuertas) del diagrama:

1. ¿El CUIT es válido y no está registrado? Si el formato es incorrecto, el bot vuelve a pedirlo; si el CUIT ya existe, rechaza el alta para evitar duplicados.
2. ¿El monto estimado anual supera el umbral de \$500.000? Si no lo supera, el alta es automática; si lo supera, se deriva a aprobación.
3. ¿El Responsable de Compras aprueba la solicitud? Si aprueba, el proveedor queda Activo; si no, queda Rechazado.

5. MODELADO DEL PROCESO EN BPMN 2.0

El siguiente diagrama representa el flujo del bot usando la notación BPMN 2.0. Se utiliza un pool con tres carriles (lanes) para diferenciar claramente quién realiza cada tarea: el Usuario/Proveedor, el Sistema/Bot y el Responsable de Compras. El proceso tiene un evento de inicio y varios eventos de fin (alta confirmada o rechazo), y las tres compuertas exclusivas (XOR) que representan las decisiones descritas.

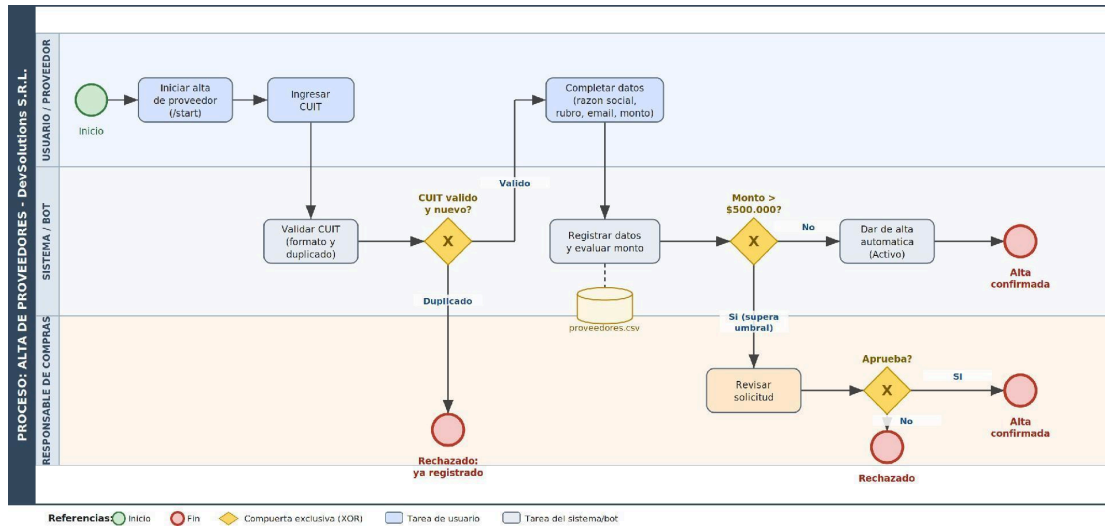


Figura 1. Diagrama BPMN 2.0 del proceso de alta de proveedores

La coherencia entre el diagrama y el código es directa, cada compuerta del BPMN se implementa en el programa con una estructura condicional (if / elif / else), y cada tarea de servicio (como “validar CUIT” o “registrar datos”) corresponde a una función del simulador que consulta o escribe en la base de datos.

6. PERSPECTIVA TÉCNICA

6.1 Plataforma y lenguaje

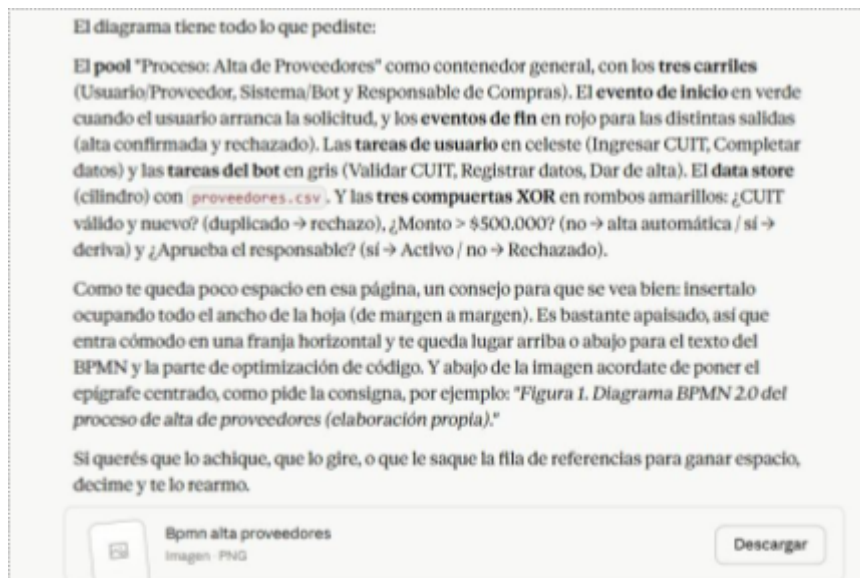
Utilizamos Python que es el lenguaje que venimos utilizando en la carrera. Para una implementación real, las plataformas posibles serían Telegram (con la librería python-telegram-bot), WhatsApp Business API o una aplicación web. Como la consigna lo permite, en este trabajo presentamos un simulador por consola que ejecuta exactamente el mismo flujo del BPMN, para concentrarnos en lo que se evalúa, la coherencia entre el modelo y la lógica.

6.2 Herramientas de IA utilizadas

Durante el desarrollo utilizamos inteligencia artificial como apoyo para realizar el gráfico de la notación BPMN dándole las instrucciones de cómo hacerlo, y para revisar el código. Justificamos su uso porque nos permitieron ser claros y prolijos, como también optimizar y detectar errores, pero todas las decisiones de diseño (qué proceso elegir, qué reglas aplicar y qué validaciones incluir) las tomamos nosotros. A continuación incluimos las capturas de pantalla de las consultas realizadas y sus respuestas:

Herramienta: Claude

Prompt utilizado: "a partir de la descripción del proceso de alta de proveedores, generá el diagrama BPMN 2.0 con sus compuertas de decisión". El diagrama obtenido se revisó para confirmar que representara correctamente el flujo del proceso (validación del CUIT, evaluación del monto y aprobación), verificando que coincidiera con la lógica implementada en el código.



Herramienta: Gemini

Prompt utilizado: "optimizá este código para que sea más robusto y maneje cualquier error que un usuario pueda ingresar". La herramienta propuso varias mejoras, pero no todas resultaron adecuadas:

Se aceptaron cambios que aportaban robustez como la limpieza de espacios en los datos ingresados (`.strip()`) y la verificación de que el archivo CSV tuviera la columna esperada antes de leerla.

Se rechazó la validación del CUIT mediante el algoritmo del dígito verificador (módulo 11), por dos motivos: dejaba inválidos los datos de prueba ya cargados en la base, y excedía el alcance del trabajo.

Se corrigió un error de la propuesta: usaba `IOException`, que no existe en Python y se reemplazó por `OSError`, que es la forma correcta.

🔧 ¿Qué optimizamos y por qué?

- Robustez en la validación del CUIT:** Tu función actual `cuit_valido` solo cuenta que tenga 11 dígitos. Para que sea un sistema profesional, implementé la validación real mediante el **algoritmo del dígito verificador (módulo 11)**. Si meten un CUIT con 11 números inventados (como `11111111111`), el sistema se dará cuenta.
- Normalización del Monto:** Al formatear el monto como string con dos decimales (`"{:,.2f}"`) en el estado `PEDIR_MONTO`, luego al evaluar el estado `EVALUAR_MONTO` intentabas pasarlo a `float`. Si bien funciona, es más limpio manejarlo internamente como número entero (`int`) o flotante durante el flujo y darle formato visual recién al guardar o mostrar en pantalla.
- Manejo Seguro de Archivos:** Modifiqué la lectura del CSV para que si el archivo está vacío o corrupto, no rompa la aplicación por falta de columnas.
- Prevención de bucles infinitos por strings vacíos:** Aseguramos que los ingresos de texto limpien bien los espacios ocultos (con `.strip()`) antes de validar.

```
# -----
# FUNCIONES DE LA BASE DE DATOS (persistencia)
# -----

def cuits_registrados():
    """Devuelve una lista con todos los CUIT que ya estan cargados de forma segura."""
    lista = []
    if not os.path.exists(ARCHIVO):
        return lista
    try:
        with open(ARCHIVO, newline="", encoding="utf-8") as f:
            lector = csv.DictReader(f)
            # Validamos que el archivo tenga datos y la columna 'cuit' exista
            if lector.fieldnames and "cuit" in lector.fieldnames:
                for fila in lector:
                    if fila.get("cuit"):
                        lista.append(fila["cuit"].strip())
    except (IOException, KeyError):
        # Si hay un error de lectura o el archivo esta corrupto, devolvemos lo que t
        pass
    return lista
```

Herramienta: ChatGPT

Prompt utilizado: “Necesito que: Optimices el rendimiento del código. Simplifiques la lógica cuando sea posible. Apliques buenas prácticas de programación. Mejores la legibilidad y el mantenimiento. Elimines código redundante o innecesario. Detectés posibles errores, bugs o casos no contemplados. Agregues comentarios técnicos claros donde sea necesario. Mantengas exactamente la misma funcionalidad original. cada mejora realizada y por qué la recomendas. Si existen varias alternativas, indicá cuál considerás la mejor y por qué. Además, evaluá: Complejidad temporal y espacial. Calidad de nombres de variables y funciones. Modularización del código. Validación de entradas. Manejo de errores. Cumplimiento de principios de código limpio.

Voy a revisar el archivo y devolverte una versión optimizada manteniendo la misma funcionalidad, más un análisis técnico punto por punto.

Pensó durante 57s >

Listo. Optimicé el código manteniendo la misma funcionalidad original del archivo cargado

 simulador_alta_proveedores .

Archivo optimizado: [Descargar simulador_alta_proveedores_optimizado.py](#)

Mejoras principales:

- Cambié la lista de CUIT por un `set`, así la búsqueda de duplicados pasa de $O(n)$ a $O(1)$.
- Evité leer el CSV cada vez que se valida un CUIT; ahora se lee una sola vez al inicio.
- Separé cada estado en una función propia, mejorando legibilidad y mantenimiento.
- Reemplacé el bloque largo de `if / elif` por un diccionario de acciones por estado.
- Centralicé constantes como campos del CSV, estados finales y palabra de cancelación.
- Mejoré la validación simple del email evitando varios `split`.
- Reemplacé `os.path` por `pathlib.Path`, una práctica más moderna y clara.
- Agregué comentarios técnicos y docstrings donde aportan valor.

Evaluación:

- Complejidad temporal: mejora en validación de CUIT duplicado, de $O(n)$ repetido a una sola carga inicial $O(n)$ y búsquedas $O(1)$.
- Complejidad espacial: usa un `set` adicional con los CUIT registrados, costo $O(n)$.
- Modularización: mejoró mucho; cada paso del bot quedó separado.
- Validación de entradas: se mantiene la lógica original, con leve mejora en email.
- Manejo de errores: se conserva el control ante errores de lectura del CSV.
- Código limpio: menos repetición, mejores nombres y estructura más mantenible.

       Fuentes

La IA sugirió reemplazar la estructura de condicionales if/elif por un diccionario de funciones o expresiones lambda. Sin embargo, se decidió no implementar dicha modificación, ya que se priorizó el uso de estructuras y conceptos acordes al nivel de conocimientos adquirido en la cursada, favoreciendo la comprensión y el mantenimiento del código.

Asimismo, se propuso modificar la función de registro de proveedores para almacenar los montos como valores con dos decimales. No obstante, se optó por conservar el formato original, debido a que dicho cambio respondía únicamente a una cuestión estética y no aportaba mejoras significativas en la funcionalidad ni en los requerimientos del sistema.

6.3 Persistencia de datos

Como base de datos simulada utilizamos un archivo CSV (proveedores.csv), tal como permite la consigna. El bot consulta este archivo para validar, comprueba si el CUIT ingresado ya existe antes de dar el alta. Cuando el alta se concreta, agrega un nuevo registro al final del archivo, con el estado y la fecha de alta. El archivo viene precargado con algunos proveedores para poder probar el camino de “proveedor duplicado”.

6.4 Gestión de estados (máquina de estados)

El bot tiene “memoria”, sabe en qué paso del proceso está cada usuario. Esto lo implementamos con una máquina de estados finita (FSM), igual que el ejemplo del cajero automático visto en la unidad. En cada momento el bot está en un solo estado y, según lo que ingresa el usuario, pasa (transiciona) al estado siguiente. La siguiente tabla resume las transiciones principales:

Estado actual	Entrada / evento	Estado siguiente	Acción
PEDIR_CUIT	CUIT válido y nuevo	PEDIR_RAZON	Guardar CUIT
PEDIR_CUIT	CUIT duplicado	FIN_RECHAZO	Rechazar
PEDIR_RAZON	Texto válido	PEDIR_RUBRO	Guardar razón social
PEDIR_RUBRO	Opción 1 a 5	PEDIR_EMAIL	Guardar rubro
PEDIR_EMAIL	Email válido	PEDIR_MONTO	Guardar email
PEDIR_MONTO	Monto válido	EVALUAR_MONTO	Guardar monto
EVALUAR_MONTO	Monto ≤ 500.000	REGISTRAR	Alta automática
EVALUAR_MONTO	Monto > 500.000	APROBACION	Derivar a aprobación
APROBACION	Responsable aprueba	REGISTRAR	Estado = Activo
APROBACION	Responsable rechaza	REGISTRAR	Estado = Rechazado

Tabla 1. Matriz de transiciones de la máquina de estados del bot.

6.5 Robustez: el camino infeliz

El bot está preparado para manejar errores de entrada del usuario (el “camino infeliz”). Si el dato ingresado no cumple con lo esperado, el bot no avanza de estado: avisa el problema y vuelve a pedir el dato. Por ejemplo, si se pide un número y el usuario escribe texto, el bot lo informa y repregunta. Además, el usuario puede escribir “cancelar” en cualquier momento para salir sin guardar nada. Las pruebas concretas se detallan en el punto 9.

7. DICCIONARIO DE DATOS

El diccionario de datos describe las características de cada campo que maneja el sistema (su tipo, longitud, valores permitidos y si es obligatorio). No guarda información real, sino que define cómo debe ser cada dato para que el programa funcione sin errores.

Campo	Tipo	Long.	Dominio / valores permitidos	Oblig.	Descripción
cuit	N Numérico (texto de dígitos)	11	11 dígitos exactos, único	Sí	Identificador tributario del proveedor. Es la clave primaria; no puede repetirse.
razon_social	Alfanumérico	≤ 100	Texto no vacío	Sí	Nombre legal de la empresa proveedora.
rubro	Alfanumérico (lista)	—	Hardware / Software-Licencias / Cloud / Consultoría / Insumos	Sí	Categoría del proveedor. Lista cerrada de opciones.
email	Alfanumérico	≤ 80	Debe contener “@” y un dominio	Sí	Correo de contacto del proveedor.
monto_estimado_anual	N Numérico (decimal)	—	Número mayor a 0, dos decimales	Sí	Monto anual estimado de contratación. Define si requiere aprobación.
estado	Alfanumérico (lista)	—	Pendiente / Activo / Rechazado	Sí (sistema)	Estado del proveedor. Lo asigna el bot según las reglas.

fecha_alta	Fecha	—	Formato AAAA-MM-DD	Sí (sistema)	Fecha en que se registró el proveedor. La completa el sistema.
------------	-------	---	--------------------	--------------	--

Tabla 2. Diccionario de datos de la entidad Proveedor.

8. MANUAL DE USUARIO

A continuación se describe cómo usar el bot:

1. Iniciar el bot ejecutando el programa `simulador_alta_proveedores.py` (o escribiendo `/start` en una versión real).
2. Responder cada pregunta del bot a medida que aparece: CUIT, razón social, rubro, email y monto estimado anual.
3. Para el rubro, elegir el número correspondiente de la lista (del 1 al 5).
4. Si el monto supera el umbral, el bot deriva la solicitud y el Responsable de Compras debe responder “sí” o “no”.
5. Escribir “cancelar” en cualquier momento para salir sin guardar la solicitud.
6. Al finalizar, el bot informa el resultado (Alta confirmada, Rechazado o Cancelado). Si el alta se concreta, los datos quedan guardados en el archivo `proveedores.csv`.

9. PRUEBAS DE ESTRÉS (MANEJO DE ERRORES)

Probamos el bot con entradas incorrectas y con todos los caminos posibles del proceso, para verificar que sea robusto. Los resultados se resumen en la siguiente tabla:

Entrada del usuario	Respuesta esperada del bot	Resultado
CUIT con letras o longitud incorrecta (ej: "456")	Avisa que debe tener 11 números y vuelve a pedirlo	Correcto
CUIT que ya existe (ej: 20304050607)	Rechaza el alta: "proveedor ya registrado"	Correcto
Rubro fuera de la lista (ej: "6" o "hola")	Pide elegir una opción entre 1 y 5	Correcto
Email sin "@" (ej: juangmail.com)	Avisa que el email no es válido y repregunta	Correcto
Monto con texto o cero (ej: "mil" o "0")	Pide un número entero positivo	Correcto
Razón social vacía (Enter sin escribir)	Avisa que no puede quedar vacía	Correcto
Monto ≤ 500.000	Alta automática, proveedor queda Activo	Correcto
Monto > 500.000 y responsable aprueba	Alta confirmada, proveedor queda Activo	Correcto
Monto > 500.000 y responsable rechaza	Proveedor queda Rechazado	Correcto
"cancelar" en cualquier paso	Termina el proceso sin guardar nada	Correcto

Tabla 3. Pruebas de robustez del bot (camino infeliz y caminos felices).

10. CONCLUSIÓN

Con este trabajo logramos integrar la mirada de negocio con la implementación técnica. Primero entendimos y modelamos un proceso real de una organización (el alta de proveedores) usando BPMN 2.0, y luego lo traducimos a un programa que sigue ese flujo. La máquina de estados le da "memoria" al bot, las validaciones lo hacen robusto frente a errores, y el archivo CSV le da persistencia a la información.

Como futuros técnicos en programación, el trabajo nos mostró que el valor de programar no está solo en escribir código, sino en entender el proceso de la empresa antes de automatizarlo. El diagrama BPMN funcionó como un "contrato" entre lo que la empresa necesita y lo que el software hace.

11. BIBLIOGRAFÍA Y MATERIAL DE REFERENCIA

- Material de cátedra de Organización Empresarial (TUPaD – UTN)
- Object Management Group (OMG). Business Process Model and Notation (BPMN) 2.0.
- Documentación oficial de Python: <https://docs.python.org/es/3/>
- Bizagi Modeler (herramienta de modelado BPMN): <https://www.bizagi.com/>

ANEXO: REPOSITORIO Y ENTREGABLES

El código fuente completo (simulador, base de datos de ejemplo y README con instrucciones) se encuentra en el repositorio del proyecto:

- <https://github.com/martufn/Alta-de-Proveedores.git>